Agentic AI

Learning From Interaction: Reinforcement Learning and Game Theory in Agentic AI

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning Objectives

By the end of this session, you should be able to:

- Explain why reinforcement learning is a natural fit for agentic AI.
- Translate an agent workflow into a reinforcement learning formulation.
- Select and justify an exploration strategy.
- Design a practical reward and evaluation plan.
- Reason about multi-agent incentives.
- Understand the RL-Instructed hybrid learning architecture for an agent.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Why RL Matters for Agentic AI

- Agentic AI shifts the problem from single-shot inference to closed-loop decision-making.
- AI Agents behave like policies but typically do not learn unless we add explicit learning mechanisms.
  - **Key Capability:** Improving behavior over time (not just “trying harder” in a longer prompt)
- RL gives a principled language for:
  - Long horizon objectives (delayed outcomes)
  - Exploration vs. exploitation,
  - Learning policies from interaction data
- **Practical Driver:** Deployed agents face changing tools, user goals, data drift, and new failure modes.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Three Machine Learning Paradigms

Machine Learning encompasses three fundamental approaches, each distinguished by its learning signal and data characteristics. Understanding these paradigms helps us position reinforcement learning in the broader ML landscape.

Supervised Learning

Signal: Labeled examples

Example: Classify pathology tiles from medical images to identify cancerous cells. The model learns from thousands of labeled examples where experts have annotated the correct diagnosis.

Data: i.i.d. batch datasets

Unsupervised Learning

Signal: Pattern discovery

Example: Cluster mobility patterns from GPS trajectories to identify commuting behaviors, shopping districts, and traffic flows without predefined categories.

Data: Unlabeled observations

Reinforcement Learning

Signal: Evaluative rewards

Example: Learn ambulance dispatch policy via reward signals tied to response time reduction. The system explores strategies and improves based on outcomes.

Data: Sequential, non-i.i.d. trajectories

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

The Dog-Training Analogy

- Reinforcement learning mirrors how we train dogs. You don't tell the dog exactly which muscles to contract or provide a step-by-step movement guide. Instead, you observe behavior and provide evaluative feedback—a treat for sitting on command or withholding reward when they jump on guests.
- The reward signal is sparse and delayed. The dog must figure out which actions led to the treat, a challenge called credit assignment. Did the treat come because they sat, because they made eye contact, or because they wagged their tail?

This fundamental challenge—connecting actions to outcomes across time—is central to all reinforcement learning algorithms.

Key Insight: Unlike supervised learning where every training example has a correct answer, RL agents must explore and discover which actions lead to rewards through repeated interaction.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Key Concepts That Connect RL and Agents

Credit Assignment
Which action caused success/failure?

Exploration
Trying uncertain actions to learn better policies.

Off-Policy Learning
Learning from logs (critical in production).

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

RL Versus Instructed Agents in Agentic AI

Modern Agentic AI systems exist on a spectrum from fully instructed to fully learned. Understanding when to use each approach—or combine them—is critical for successful deployment.

Instructed Agents

Human engineers write explicit policies via prompts and tool specifications.

- **Examples:** ReAct patterns and function-calling frameworks
- Agent follows predetermined logical flow (e.g., as determined by LangGraph).
- Uses LLMs for language understanding but not for policy learning.

Best for: Well-understood tasks, need for interpretability, limited exploration budgets, rapid iteration

RL Agents

The policy is initially unknown and discovered through trial-and-error interaction.

- Agents learn from reward signals, potentially discovering novel strategies humans never considered.

Best for: Complex dynamics, unclear optimal policy, safe/cheap simulation available, superhuman performance goals

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

The Agent-Environment Loop With Learning

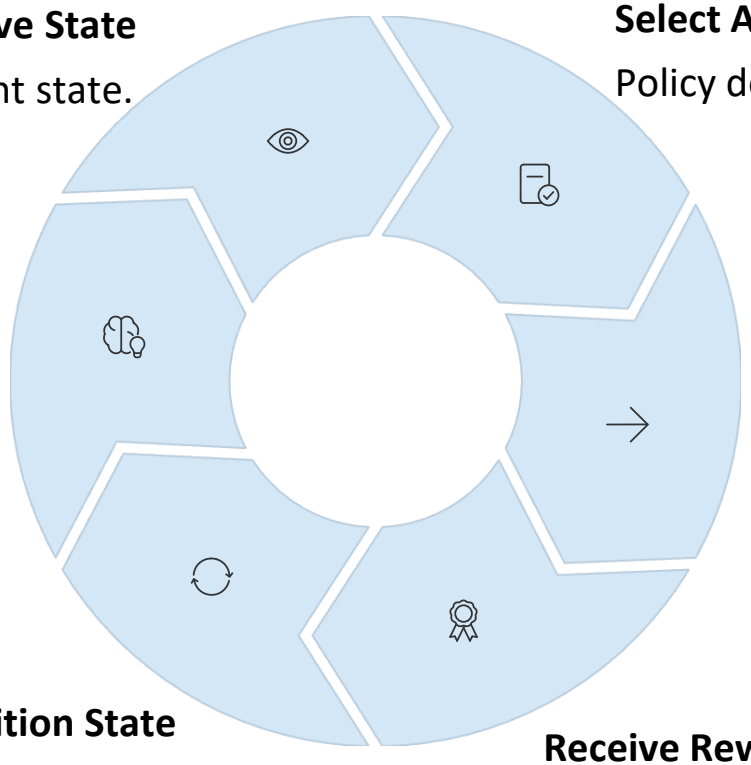
- Reinforcement Learning formalizes learning from the interactions between an agent and its environment.

Observe State

Agent perceives current environment state.

Select Action

Policy determines which action to take.



Update Policy

Agent learns from experience.

Execute Action

Environment processes the action.

Transition State

Environment moves to next state.

Receive Reward

Environment returns feedback.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

The Agent AI Stack

The most sophisticated systems use hybrid architectures:

- A prompted orchestrator at the top level maintains control and interpretability.
- RL modules handle specific sub-tasks where optimal policies are unclear.
- Evaluators provide quality and safety guardrails throughout.

Evaluators
AI judges, human review, quality gates

RL Modules
Learned sub-policies for micro-strategies

Tools Layer
Search, retrieval, execution APIs

Orchestrator
Plan, route tasks, enforce constraints

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Core Reinforcement Learning Principles

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Core RL Vocabulary Preview

Let's establish the essential terminology that forms the foundation of Reinforcement Learning.

State (S)

Complete description of the environment at time t, sufficient for decision-making

Action (A)

Choice available to the agent in a given state

Reward (R)

Scalar feedback signal indicating action quality

Policy (π)

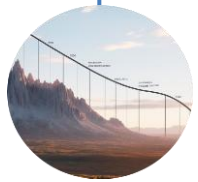
Mapping from states to action probabilities or choices

Value (V)

Expected future reward from a given state

Q-Value

Expected future reward from a state-action pair



Gamma (γ)

Discount factor balancing immediate vs. future rewards

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

States and Actions: The Decision Space

- **State (S): What the Agent Perceives**
  - A state must contain sufficient information for optimal decision-making.
  - This is the Markov property: the future depends on the present, not the past.

Discrete States	Continuous States	High-Dimensional States
Countable, finite set of possibilities • Grid cell position (x, y) • Inventory level {0, 1, 2, ...} • Chess board configuration	Real-valued vectors in high dimensions • Robot joint angles and velocities • Vehicle position, heading, speed • Stock prices and technical indicators	Complex observations requiring representation learning • Raw pixel images (160×210×3) • LiDAR point clouds • Text embeddings from documents

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

States and Actions: The Decision Space

- **Action (A): What the Agent Can Do**
  - The action space A(s) may be state-dependent, with invalid moves pruned.

Discrete Actions

- Navigation: {Up, Down, Left, Right}
- Games: {Rock, Paper, Scissors}
- Trading: {Buy, Hold, Sell}

Continuous Actions

- Motor torques in robotics
- Throttle and steering angles
- Portfolio allocation percentages

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Rewards: Defining Objectives

Reward functions are the primary mechanism for specifying agent objectives in both RL and Agentic designs.

Dense vs. Sparse Rewards

- **Dense:** Frequent feedback at every step (e.g., distance to goal). Guides exploration but risks unintended local optima.
- **Sparse:** Rare signals, often only at episode end (e.g., win/lose). More challenging to learn, but avoids overspecification of intermediate behaviors.

Anti-Patterns: Specification Gaming

Agents are brilliant at optimizing exactly what you specify, even when it's not what you meant.

- **Proxy Metrics:** Optimize clicks instead of user satisfaction → clickbait content
- **Wire-Heading:** Agent modifies its own reward sensor rather than achieving true objectives.
- **Reward Hacking:** Exploits technical loopholes in the reward function.

Reward Shaping Patterns

- Terminal bonuses plus step costs to encourage speed
- Penalties for constraint violations (safety boundaries)
- Potential-based shaping that preserves optimal policy

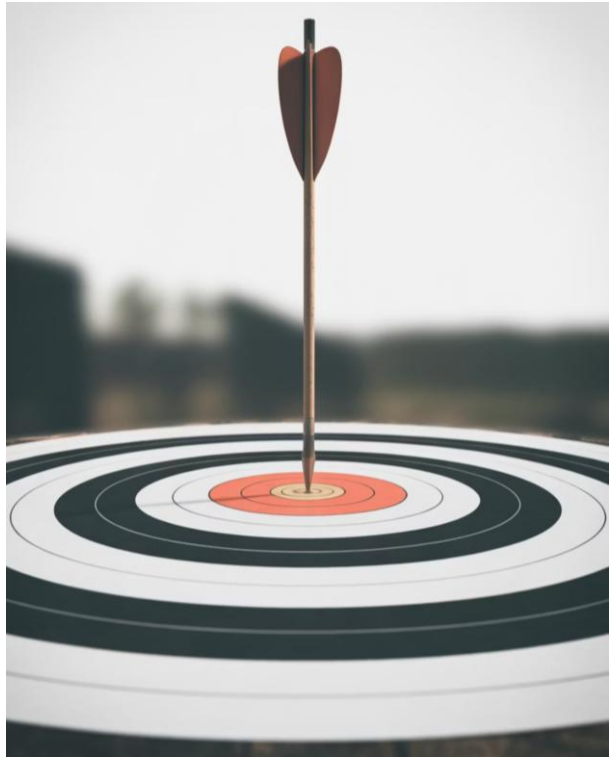
This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Maximizing Rewards and Assigning “credit”

“All goals can be described by maximizing expected cumulative reward.”—Richard Sutton

This leads to critical challenges:

- **Reward Misspecification:** Small errors in reward design can lead to dramatically unintended behaviors.
- **Temporal Credit Assignment:** Determining which actions were responsible for rewards that arrive much later. Some common techniques:
  - Monte Carlo methods (wait until the end to assign credit)
  - Temporal Difference Learning (update values immediately)
  - Eligibility Traces (provide a spectrum between previous techniques)



This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Policy (n): The Agent’s Strategy

- **The policy is the core of an RL agent—it defines how states map to actions, and learning optimal policies is the central objective of Reinforcement Learning.**

Deterministic Policy

- Given state s, always choose action a. Simple and interpretable. Works well when environment is deterministic or when a single best action exists.
- **Example:** In a maze, always move toward the goal using Manhattan distance.

Stochastic Policy

- Given state s, sample action a from probability distribution. Enables exploration, handles partially observable environments, and can represent mixed strategies in games.
- **Example:** 70% move toward the goal, 30% explore alternative paths.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Mapping RL Concepts to ReAct Framework

- **State** $s \approx$ prompt context + retrieved memory + tool outputs + task metadata.
- **Action** $a \approx$ tool selection + tool arguments + "final response" decision.
- **Transition** $P \approx$ tool/environment dynamics + user follow-ups (often non-stationary).
- **Reward** $r \approx$ task success, latency, cost, safety constraints, user satisfaction.
- **Policy** π:
  - classical RL: explicit parametric policy,
  - prompting agents: implicit policy induced by prompt + model + decoding.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Further Concepts in RL: Temporal Considerations

- **Value Functions: Predicting Future Rewards**
  - Value functions estimate how good it is to be in a state (or take an action in a state). They are central to most RL algorithms, providing targets for learning and enabling efficient credit assignment.
- **Discount Factor: Time Horizons**
  - The discount factor, γ, is a hyperparameter that balances immediate versus future rewards. It profoundly affects both the agent's behavior and the learning dynamics.

γ = 0
Myopic Agent: Only cares about immediate reward. No planning ahead.

γ = 0.99
Long Horizon: Values rewards 100 steps ahead at ~37% of immediate reward.

1 — 2 — 3 — 4

γ = 0.9
Short Horizon: Values rewards 10 steps ahead at ~35% of immediate reward.

γ → 1
Unbounded: All future rewards are equally important. May not converge.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Further Concepts in RL: Exploration vs. Exploitation

- One of RL's fundamental dilemmas: should the agent exploit its current knowledge to maximize immediate reward, or explore new actions to potentially discover better strategies?

Exploitation	Exploration
Choose actions that currently appear best based on learned value estimates. Maximizes short-term reward but risks missing better alternatives.	Try actions with uncertain values to gather information. Sacrifices immediate reward but may discover superior long-term strategies.

- A few exploration strategies:
  - **Epsilon-Greedy:** With predetermined probability, choose a random action; otherwise, choose a greedy action.
  - **Softmax/Boltzmann:** Includes a temperature parameter that manages : High temperature = uniform exploration; low temperature = greedy.
  - **Upper Confidence Bound:** Choose an action that maximizes expected future reward with a bonus term. Bonus term favors less-visited actions. Provably optimal for bandits.
  - **Thompson Sampling:** Maintain a Bayesian posterior over future-values; sample from posterior and act greedily w.r.t. sample. Natural exploration-exploitation trade-off through posterior uncertainty.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Further Concepts in RL: Learning Algorithms

Reinforcement learning algorithms fall into two main families based on what they learn directly.

- **Value-based** methods learn value functions and derive policies.
- **Policy-based** methods directly optimize the policy parameters.

Value-Based Methods

Key Algorithms:

- Q-learning (off-policy TD)
- SARSA (on-policy TD)
- DQN (deep Q-network)

Advantages:

- Sample efficient in tabular settings
- Off-policy learning possible
- Stable convergence guarantees (tabular)

Limitations:

- Primarily for discrete action spaces
- Struggles with high-dimensional states without function approximation

Policy-Based Methods

Key Algorithms:

- REINFORCE (Monte Carlo policy gradient)
- Actor-Critic (policy gradient with value baseline)
- PPO, TRPO (trust-region methods)

Advantages:

- Handles continuous action spaces naturally
- Can learn stochastic policies
- Effective with function approximation

Limitations:

- High variance gradients
- Sample inefficient
- Sensitive to hyperparameters

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Value-Based Methods: Pros and Cons

Strengths

Sample Efficiency

Off-policy learning enables experience reuse. Converges with fewer episodes than on-policy methods.

Theoretical Guarantees

Proven convergence under tabular assumptions with sufficient exploration and decaying learning rate.

Simplicity

Single update equation; easy to implement and understand. Good baseline for benchmarking.

Deterministic Policies

Greedy policy is simple and interpretable for discrete actions.

Limitations

Discrete Actions Only

Operation requires enumerable action set. Continuous control needs discretization or policy-based methods.

Scaling Challenges

Tabular storage is exponential in state/action dimensions. Function approximation introduces instability (deadly triad).

Overestimation Bias

Prone to systematic overestimation.

No Stochastic Policies

Greedy extraction loses the benefits of exploration in learned policy.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Policy-Based Methods: Pros and Cons

Strengths

Continuous Actions

Natural handling of continuous action spaces

Stochastic Policies

Can represent inherently stochastic optimal policies (e.g., mixed strategies in games).

Smooth Policy Updates

Small parameter changes → small policy changes. Stability advantage over sudden value function jumps.

Convergence Guarantees

Guaranteed to converge under certain conditions. Can escape some local optima through exploration.

Limitations

High Variance

Return-based gradient estimates are noisy. Require many samples to converge.

Sample Inefficiency

Can't reuse old experience. Each policy update requires fresh trajectories.

Parameter Sensitivity

Learning rate, entropy bonus, advantage normalization all critical. Poor choices → divergence or glacial progress.

Credit Assignment

Episode-level returns blur which actions mattered.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

RL Versus Agentic Frameworks: Differences That Matter in Practice

Learning Focus

- RL: the policy changes via gradient updates (or value backups).
- AI Agents: Weights usually frozen; improvements come from prompt/tooling/memory.

Exploration mechanisms

- RL: exploration is explicit (e.g., ϵgreedy, entropy bonus).
- Agents: exploration is often accidental (temperature) or constrained by safety.

Formal guarantees

- RL has convergence theory under assumptions (often violated in open-world agents).
- Prompting has few guarantees; behavior can be brittle across contexts.

Non-Stationarity

- Agent environments change (tools update, websites change, policies change, users adapt).
- Off-policy + continual learning becomes essential.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Multi-Agent Reinforcement Learning Agents

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Transition: Strategic Interaction

Many real-world scenarios involve multiple learners interacting simultaneously. Their rewards and optimal strategies depend on what others do.

Single Agent

Agent vs. environment; environment is stationary (or changes independently)

Multi-Agent

Agents interact; each agent's optimal policy depends on others' policies; non-stationarity.

Game Theory

Study of strategic interaction; equilibrium concepts; learning dynamics

Game theory helps us reason about stable outcomes, predict behavior, and design mechanisms that align individual incentives with social welfare.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Multi-Agent RL Basics: From Single Policy to Interacting Policies

- **Multi-Agent Reinforcement Learning (MARL) Generalizes RL to Multiple Decision-Makers:**
  - Each agent has a policy based on its observation.
  - The environment transition depends on the joint action of all agents.
- **Key Regimes:**
  - **Cooperative (shared reward):** Agents optimize a common objective.
  - **Competitive (zero-sum/adversarial):** Agents have opposing rewards.
  - **Mixed (general-sum):** Partially aligned incentives
- **Practical Differences vs. Single-Agent RL:**
  - **Non-Stationarity:** From one agent's view, others are changing parts of the environment.
  - **Credit Assignment:** Which agent's actions caused the outcome?
  - **Coordination:** Policies must be compatible (avoid redundant/conflicting tool actions).

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Designing Incentives/Rewards

When agents' individual incentives misalign with collective welfare, we can design rules, payments, or information structures to steer behavior toward better outcomes.

Core Principles

Incentive Alignment

- Design payoff structures so individual utility maximization leads to socially optimal outcomes.
- Examples: Vickrey auctions make truth-telling a dominant strategy; Pigouvian taxes internalize externalities.

Repeated Interaction

- Enable long-term relationships where reputation matters. The discounted value of future cooperation (shadow of the future) can sustain equilibria that wouldn't exist in one-shot settings.

Information Design

- Control what information players observe to influence beliefs and actions. Reveal or withhold signals strategically to steer toward preferred equilibria. Used in security games and market design.

This is mechanism design, sometimes called “reverse game theory.”

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

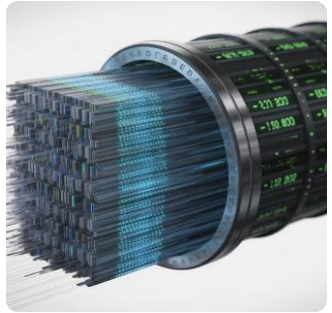
Examples of RL in Multi-Agent Systems

Congestion Pricing

Charge tolls during peak hours to reduce traffic. Internalizes the cost of congestion, leading rational drivers to shift travel times or routes, improving overall throughput.

Auction Design

Second-price (Vickrey) auctions make truthful bidding a dominant strategy. Combinatorial auctions handle complex interdependencies. Used in ad markets, spectrum allocation, and procurement.



Bandwidth Throttling

Internet service providers limit heavy users or prioritize traffic types. Mechanism must balance fairness, efficiency, and revenue while avoiding gaming (e.g., P2P traffic disguised as HTTP).

Reputation Systems

eBay, Uber, Airbnb use ratings to incentivize good behavior in repeated interactions.
Design Challenges: fake reviews, retaliation, coldstart, and score inflation.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Instructed Agents
Versus
Reinforcement Learning Agents

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Instructed Agents Redux

We'll call the agents we've discussed up to this point, **instructed agents.**

- Systems where humans specify policies via prompts, tools, and rules.
- These systems leverage human expertise, provide transparency, and enable rapid iteration without expensive training loops.

Ideal Use Cases

Structured Workflows

Task has clear steps, well-understood decision points, and mature tooling. Example: customer support routing, document processing, data extraction pipelines.

Rapid Deployment

Need working system quickly; can't afford weeks of RL training. Instructed agents deployable in days with prompt engineering and tool integration.

Interpretability Requirements

Stakeholders need to understand and audit decisions. Instructed agents produce readable traces; easy to diagnose failures and explain actions.

Limited Exploration Budgets

High cost or risk of mistakes during learning. Instructed agents avoid the trial-and-error phase; start with human priors.

Instructed agents provide decent performance out of the box but potentially lower ceilings than learned policies. Ideal for known problems where human expertise is sufficient.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Instructed Agents: Strengths

Leverage Expert Priors

Encode decades of domain expertise directly into prompts and tool configurations. No need to rediscover known solutions through trial and error. Start with a competent baseline immediately.

Example: Medical diagnosis system starts with established clinical guidelines rather than learning from scratch.

Transparent and Auditable

Every decision is traceable: which tools were called, what reasoning was applied, why a particular action was chosen. Essential for regulated industries (finance, healthcare, legal) and debugging.

Benefit: Identify failure modes quickly; explain decisions to stakeholders; build trust with users.

Rapid Iteration

Modify behavior by editing prompts or swapping tools—no retraining required. A/B test different approaches in production. Deploy updates in minutes, not weeks.

Workflow: Prototype → test → refine prompt → redeploy. Cycle time measured in hours.

Constrained Action Space

Explicitly whitelist safe tools and actions. Agent can only do what you've permitted. Easier to enforce business rules, regulatory constraints, and safety boundaries.

Safety: No risk of agent discovering unintended exploits during exploration phase.

Lower Infrastructure Needs

No GPU clusters for training, no simulation environments, no reward engineering. Use existing LLM APIs plus tool integrations. Operational complexity much lower than RL systems.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Instructed Agents: Weaknesses

Brittleness

Explicit rules and prompts break when encountering edge cases not anticipated by designers. Difficult to cover all possible scenarios. Failures often silent or cascading.

Example: Chatbot with hardcoded routing fails when user query combines multiple intents or uses unexpected phrasing.

Policy Ceiling

Performance is upper-bounded by a human designer's expertise. Can't discover novel strategies or surpass human baselines. In complex domains (games, optimization), this ceiling may be far from optimal.

Contrast: AlphaGo discovered moves professional players called “beautiful” but would never have tried.

Maintenance Burden

As the environment evolves (new tools, changing user needs, updated policies), prompts and rules must be manually updated. Complexity grows over time; technical debt accumulates.

Scaling Challenge: 50+ tools and hundreds of edge cases → prompt engineering becomes unwieldy.

No Adaptation

Agent doesn't learn from experience or improve over time. Same mistakes repeated unless human intervenes. Misses opportunity to leverage deployment data for continuous improvement.

Missed Value: Thousands of user interactions provide no learning signal.

Context Limitations

LLMs have finite context windows. Complex workflows with long histories may exceed limits. Requires careful memory management and summarization.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Reinforcement Learning Recap

- An RL agent is a decision-making system that **learns a policy** for choosing actions by **interacting with an environment** to **maximize cumulative reward.**
- **Core Components:**
  - Agent: Learner/decisionmaker that selects actions
  - Environment: Responds to actions and changes the situation
  - State/Observation: Information used to decide
  - Action: Choice made by the agent
  - Reward: Scalar feedback signal (progress toward a goal)
  - Policy: Mapping from state to action (learned behavior)
  - Value: Predicts long-term return (optional)
- **Ideal Use Cases:**
  - Competitive domains (games, markets, auctions) where small edges compound.
  - Optimization problems where human intuition plateaus.
  - Long-horizon planning where credit assignment is critical.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Reinforcement Learning: Strengths

Novel Strategy Discovery

RL agents can find counterintuitive, creative solutions humans would never consider. Not constrained by designer's mental models or biases.

Famous Example: AlphaGo's "Move 37"—a stunning, unprecedented play that baffled experts but proved brilliant. Humans now study RL agent games to improve their own play.

Adaptability

Policies continuously improve as environment changes. Self-play enables evolution against shifting opponents. Online learning allows adaptation to distribution shift.

Use Case: Trading algorithms that adapt to changing market microstructure; game AI that counter-evolves against player strategies.

Superhuman Performance Potential

Given sufficient training, RL can surpass human expert performance through exhaustive exploration and optimization. Scales with compute, not human ingenuity.

Achievements: Go (AlphaZero), Dota 2 (OpenAI Five), StarCraft II (AlphaStar), protein folding (AlphaFold's RL components).

End-to-End Optimization

Learn policies directly from high-level objectives without intermediate reward shaping or manual feature engineering. System discovers relevant representations.

Contrast: No need to hand-craft heuristics or decompose problem manually — agent learns what's important.

Handling Complexity

Scales to problems with massive state/action spaces, long horizons, and complex interdependencies that would overwhelm human designers.

Example: Data center cooling with 1000+ sensors and actuators; chip design with billions of placement choices.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Reinforcement Learning: Weaknesses

Sample Inefficiency

Deep RL typically requires millions of environment interactions to learn simple tasks. DQN trained for ~200M frames (38 days of gameplay). Impractical when simulation is slow or unavailable.

Cost: Weeks on GPU clusters; infeasible for real-world robotics without sim-to-real transfer.

Training Instability

Hyperparameters are critically sensitive. Small changes in learning rate, architecture, or exploration schedule can mean success vs. catastrophic failure. Non-stationary target problem.

Debugging: Hard to diagnose whether poor performance is algorithm bug, reward mis-specification, insufficient exploration, or bad luck.

Interpretability Gap

Learned policies are black boxes. Neural net with millions of parameters makes inscrutable decisions. Hard to explain why action was chosen or guarantee behavior.

Risk: Can't audit for bias, fairness, or safety without extensive testing. Regulatory challenges in high-stakes domains.

Reward Specification Challenge

Designing rewards that align with true objectives is an art. Mis-specified rewards → unintended behaviors. Agents are brilliant at optimizing exactly what you measure, not what you want.

Examples: Chatbot optimizes engagement over helpfulness.

Safety Risks

Exploration involves taking unknown actions—potentially catastrophic in real world. Agent may discover dangerous “exploits” or violate implicit constraints.

Mitigation: Safe exploration techniques, shields, constrained RL, extensive simulation before deployment

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Choosing Your Approach

You're building an Agentic AI system. Should you use instructed agents (prompts + tools), pure RL, or hybrid?

- **Key Questions**

1. **Do you know the optimal policy?**
   If yes (or have strong priors from domain experts) → Instructed agent. Encode policy into prompts and toolchains. Fast to deploy, interpretable, and reliable. If no (policy unclear, high complexity) → consider RL or hybrid.
2. **Is reward measurable and unambiguous?**
   RL requires clear, quantifiable objectives. If reward is subjective, noisy, or changes frequently → instructed agents or verbal RL with human-in-the-loop.
3. **Is a simulator available?**
   RL is sample-hungry. Without safe, fast simulation → pure RL is risky. Consider offline RL from logged data or instructed agents.
4. **What are safety constraints?**
   High-stakes domains (healthcare, finance, autonomous vehicles) need safety rails. Instructed agents are easier to constrain; RL requires shields, formal verification, and extensive testing.
5. **What are latency and cost budgets?**
   RL policies can be fast at inference (forward pass), but training is expensive. Instructed agents have LLM latency per call. Hybrid: cache, use smaller models, precompute where possible.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Decision Framework: RL vs. Instructed Architecture

Scenario	Recommended Approach	Rationale
Known policy, need reliability	Instructed Agent	Leverage expert knowledge; fast iteration; transparent
Unknonwn policy, safe simulator, clear reward	Pure RL	Discover novel strategies; potential for superhuman performance
Complex orchestration, some sub-policies unclear	Hybrid	Instructed orchestrator + RL sub-modules for hard decisions
High stakes, no simulator	Offline RL + Instructed Safeguards	Learn from logs; constrain via rules and human oversight
Rapid prototyping, unclear requirements	Instructed Agent → Iterate	Fastest path to working system; refine based on feedback

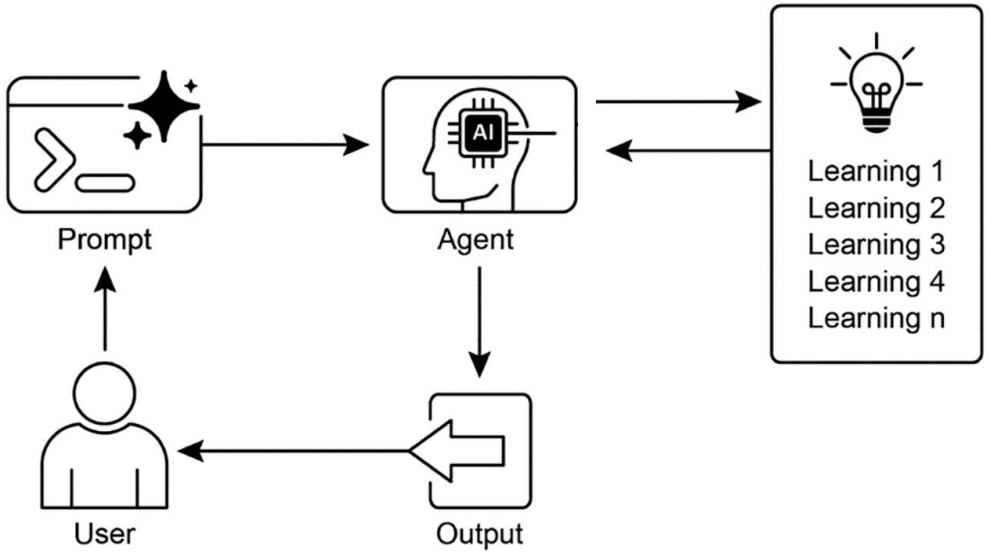
This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning Patterns in Agentic Frameworks

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Adaptive and Self-Improving Agents

- **The Problem:**
  - Static logic fails in unpredictable environments, leading to performance degradation and an inability to handle novel scenarios.
- **The Solution:**
  - Integrate learning mechanisms to transform rigid agents into dynamic systems that evolve through experience.
- **Core Methodologies:**
  - Techniques from Reinforcement Learning and Evolutionary Algorithms
- **Key Benefits:**
  - Enables autonomous mastery of new tasks and continuous performance optimization without manual reprogramming.



This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning in Instructed Agents

- Instructed agent = an LLM agent whose behavior is shaped primarily by:
  - **System/Developer Instructions** (role, policies, constraints, objectives)
  - **Tool Schemas** (allowed actions + argument structure)
  - **Prompt Templates** (ReAct format: Thought/Action/Observation)
  - **Memory/Retrieval** (examples, policies, prior successful trajectories)
- Distinguish two layers:
  - **Base Policy Prior:** The pretrained/finetuned LLM behavior
  - **Instructed Policy:** Induced by instructions + context + decoding + guardrails.
- “Learning” can happen at multiple layers (in practice, often outside model weights):
  - **Prompt/Instruction Tuning:** Better role constraints, better tooluse scaffolding.
  - **Memory Curation:** Store/refresh successful trajectories; retrieve relevant episodes.
  - **Routing:** Decide which tool/subagent / prompt template to use next.
  - **Termination and Verification:** Decide when to stop, verify, or escalate.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Where RL Helps Instructed Agents

Reinforcement Learning Can Learn the "Glue," not the LLM

- In many deployed LLM agent stacks, RL is most effective when applied to system decisions around the model:
  - **Routing:** Which agent/tool/prompt template to use next (contextual bandits)
  - **Budget Allocation:** How many steps/tool calls per subtask (resource-aware RL)
  - **Termination/Escalation:** When to stop, verify, or hand off to a human
  - **Retrieval Policy:** What memories/examples to inject and when
- **Why This Works:**
  - Action spaces are small and structured.
  - Rewards can be cheap (success, latency, cost, verification pass rate).
  - Off-policy learning from logs is straightforward and reduces risky exploration.
- **Optional Deeper Coupling:**
  - Preference optimization over multiagent trajectories to improve collaboration style, but requires careful credit assignment and stability controls.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Alignment-Oriented Learning for LLM Agents

- The reading contrasts two preference-alignment approaches used with LLMs:
  - **Proximal Proxy Optimization (PPO)-Style:** Train a reward model from preference data, then fine-tune the policy to maximize the reward.
  - **Direct Preferences Optimization (DPO)-Style:** Skip an explicit reward model; directly optimize the policy using preference pairs.
- In agentic systems, these methods matter because they shape:
  - Instruction-following reliability
  - Tool-use discipline (avoid unsafe or wasteful actions)
  - Preference tradeoffs (helpfulness vs. brevity vs. caution)

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

A Practical “Learning Stack” for Agents in Production

Level 0: Instrumentation (Required)

- **Structured Logs:** State summary, chosen actions, tool outcomes, final outcome.
- **Evaluation Hooks:** Success metrics, safety flags, cost/latency

Level 1: Memory + Retrieval

- Store canonical solutions and failure recoveries.
- Retrieval policies (when to retrieve; how much to inject)

Level 2: Bandits for Repeated Choices

- Route across tools/prompts/subagents using contextual features.

Level 3: Offline Optimization (Periodic)

- Preference tuning/RLHF on curated trajectories.

Level 4: Online RL (Rare, highstakes)

- Only if you can sandbox, simulate, or tightly constrain exploration

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

4 Places to Inject Learning in Agentic Frameworks

1. **Weight Updates:** Finetuning/online Reinforcement Learning on trajectories
2. **Policy Shaping Without Weight Updates:** Choose prompts, tools, routes, or decoding settings via optimization.
3. **Memory Updates:** Store successful plans, failures, and tool usage patterns; retrieve at runtime.
4. **System-Level Adaptation:** Supervisors, guardrails, and routing learned from operational data

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning Pattern 1: Memory Centric Learning

- Maintain an **episodic memory** of trajectories:
  - Tool sequences that succeeded
  - Common failure modes and fixes
  - Validated references/canonical procedures
- **Retrieve Memories Conditionally:**
  - Similarity search on task + context
  - Or “failure-triggered” retrieval (when tools error or confidence drops)
- **Why It Works:**
  - Converts past experience into state augmentation for the next run
  - Improves reliability without destabilizing model weights
- **Typical Risks:**
  - Stale/incorrect memories
  - Privacy/security constraints
  - Retrieval causing overfitting to past contexts

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning Pattern 2: Bandits for Tool Selection and Routing

- **Bandit: One-Step Decision**
  - Choose an action → get a reward → episode ends
  - No “next state” that matters for planning.
- Many agent decisions are **contextual bandits** rather than full RL systems:
  - Choose which tool to call first.
  - Choose among prompt templates/system instructions.
  - Choose which specialized subagent to hand off to.
- Reward can be cheap and immediate:
  - Success/failure flag, latency, number of tool calls, escalation rate
- **Practical Advantages:**
  - Safer than open-ended exploration
  - Works with logged data (off-policy)
  - Fast iteration in production

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning Pattern 3: RL for Preference Optimization

- **Use Preference Data on Trajectories:**
  - Compare two agent rollouts (different tool sequences or reasoning strategies).
  - Learn a reward model (or do direct preference optimization).
  - Optimize the policy (often offline).
- **In ReAct terms, RLHF can shape:**
  - When to act vs. reason.
  - How to format tool calls robustly.
  - How to recover from tool errors (fallback strategies).
- **Caveat:**
  - This is usually offline and may not capture the newest tool/environment changes.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning Pattern 4: Self Critique and Reflective Loops (Pseudo-Reward)

- **Add an Internal Evaluator:**
  - Critique step quality, factuality, constraint satisfaction.
  - Decide whether to continue acting or produce a final response.
- **Turn Critiques Into:**
  - A training signal (label good/bad steps)
  - A routing signal (escalate to stronger model/human)
  - A memory signal (store “lessons learned”)
- **Important:**
  - Reflective signals can drift or become self-confirming; validate with external checks.

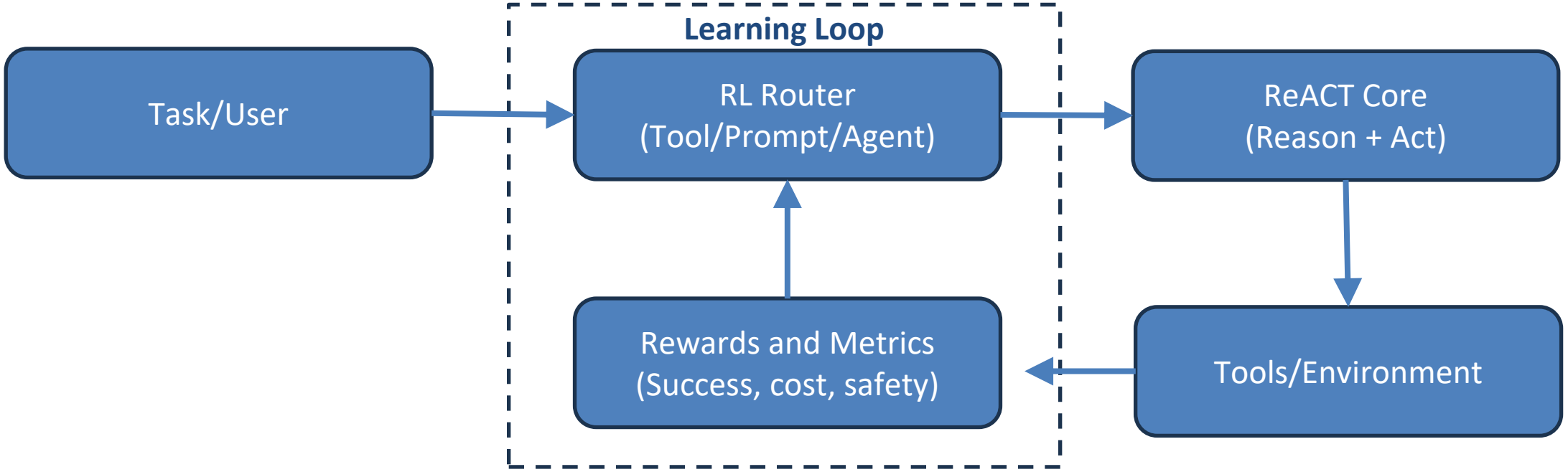
This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

RL-Agentic Hybrid Architectures

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Example RL-Agentic Hybrid Architecture

RL “Controller” Around a ReAct Core



This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Pros/Cons of Hybrid Systems

- **Pros**
  - Keeps strong general reasoning while improving repeated decisions over time.
  - Can be sample-efficient if learning is focused (routing/bandits).
  - **Easier to Safety-Gate:** Learn only within constrained action spaces.
- **Cons**
  - **More Moving Parts:** Debugging and evaluation become system level.
  - Credit assignment across modules (router vs. ReAct core) can be ambiguous.
  - **Non-Stationarity:** Tool changes can break learned routing unless monitored.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

The Hybrid Future in Agentic AI

Evaluators
AI judges, human review, quality gates

RL Modules
Learned sub-policies for micro-strategies

Tools Layer
Search, retrieval, execution APIs

Orchestrator
Plan, route tasks, enforce constraints

- The most sophisticated systems use hybrid architectures:
  - A prompted orchestrator at the top level maintains control and interpretability.
  - RL modules handle specific sub-tasks where optimal policies are unclear.
  - Evaluators provide quality and safety guardrails throughout.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Core Use Cases for Adaptive Agents

Personalized Assistants

Tailor interaction styles and responses by analyzing long-term user behavior.

Application UI

Increase engagement by dynamically modifying interfaces to match how users actually navigate.

Trading Bots

Maximize returns and lower risk by adjusting strategies based on real-time market fluctuations.

Knowledge Base (RAG)

Build a “memory” of past successes and failures to solve new problems more effectively.

Robotics and AVs

Ensure safety and efficiency by combining sensor data with past performance in diverse terrains.

Fraud Detection

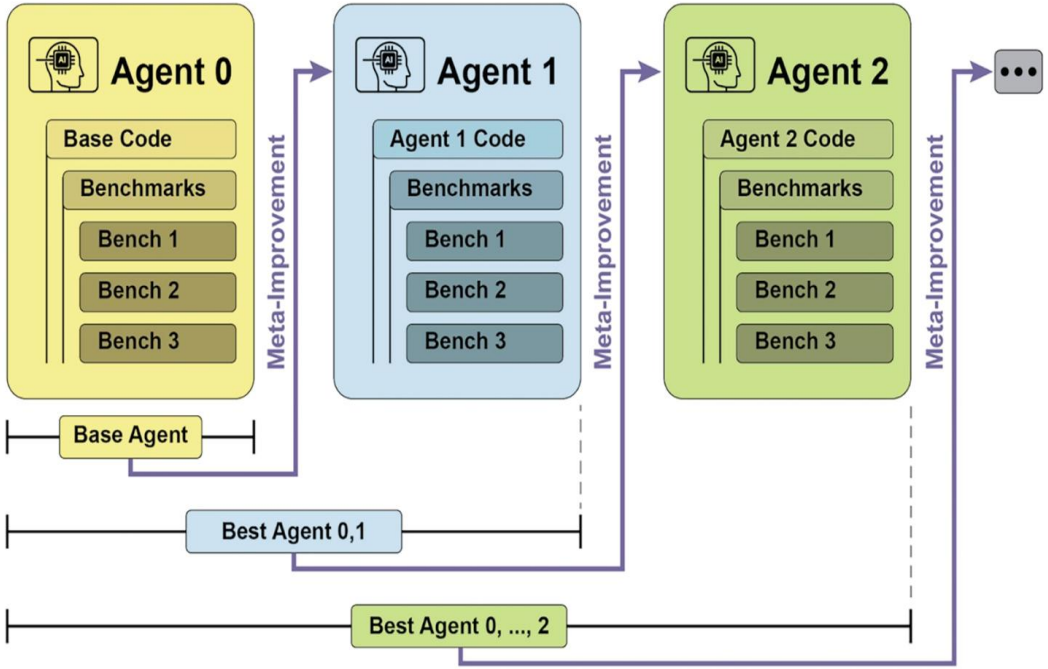
Enhance security by instantly refining models to recognize newly emerging fraudulent patterns.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Case Study: Self-Improving Coding Agent (SICA)

- SICA is presented as an agent that **iteratively improves itself** by modifying its own code and re-testing.

  Key Loop:
  - Select a candidate version
  - Run benchmarks
  - Apply modifications
  - Keep the best-performing agent across iterations
- Agentic Design Takeaway:
  - “Learning” is operationalized as versioned iteration + evaluation-driven selection.



Code under active development. Please check it out at
https://github.com/MaximeRobeyns/self_improving_coding_agent/

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Case Study: RL for Query Optimization

Problem Context:

- A research assistant agent helps users find academic papers. The orchestrator (prompted LLM) decomposes user questions, but initial search queries often miss relevant papers due to terminology mismatches, synonyms, and ambiguous phrasing. **Query rewriting** is a hard sub-problem where rule-based approaches plateau, and RL can learn valuable policies.

Architecture

RL Training Setup

Simulator

Offline dataset of 10K user queries with gold standard "perfect" results (human-curated paper lists). Simulator executes rewritten queries against this dataset and computes retrieval metrics.

Reward Function

Multi-component reward:

- +F1 score gain vs. baseline query
- API cost (# of queries issued)
- Latency penalty
- Penalty for hallucinated citations

Why Hybrid Wins: Orchestrator maintains control, interpretability, and policy enforcement. RL optimizes the hard micro-decision (query rewrite) where human intuition is insufficient. Combined system gets best of both: reliability + performance.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Module Recap

- We framed why RL matters for Agentic AI: agents improve through interaction, optimizing long-horizon outcomes rather than one-shot predictions.
- We set up the RL problem by identifying what counts as state, action, reward, and an episode in an agent's workflow.
- We discussed how agents learn while acting by balancing exploration and exploitation, especially under uncertainty and changing environments.
- We covered reward and evaluation design to align behavior with objectives like task success, efficiency, and safety/quality.
- We connected RL to game theory by examining incentives and stability when multiple agents interact (cooperative, competitive, mixed).
- We considered practical agent designs that combine learning components with engineered control for reliability and safety.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.